

Correct Optimizations: Phase 1 Report

Based on Minotaur: A SIMD-Oriented Synthesizing Superoptimizer [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

Claudia Dünzinger
Technical University Munich

Abstract

TODO

1 Introduction

This report aims to analyze and propose opportunities to continue the research from Liu et al. [1].

2 Paper Summary

2.1 Context

Minotaur is a superoptimizer that identifies missed optimization opportunities by the LLVM compiler by slicing generated IR and trying to synthesize new faster code. Its synthesizer creates improved machine code, generating and comparing equivalent but cheaper instruction sequences. To guarantee correctness of the optimization, Alive2 was extended to formally verify the generated code. Additionally, to reduce computation overhead, Minotaur also caches discovered transformations for reuse in future compilations.

The use case of Minotaur is to further optimize already heavily optimized code from a compiler which still often contains suboptimal code segments, particularly for SIMD operations. The authors showed that even LLVM’s advanced autovectorizer produced redundant or inefficient SIMD instructions. For those missed opportunities, Minotaur can correct these suboptimal instruction sequences by replacing them with improved machine code during the compilation. Some of those could be generalized and added to LLVM. Hence, the authors enabled higher performing and additionally formally verified machine code generation through the use of Minotaur.

Before Minotaur, several superoptimizer already existed that search through the code, be it assembly or LLVM IR, for more efficient sequences instead of performing set rewriting rules like traditional compilers. Common examples include Souper – with focus on scalar integer optimizations and which also uses LLVM IR – and STOKE, which uses stochastic

search for faster x86 code blocks. However, those tools only addressed part of the problem. Souper [2] detects only improvements for integer operations, but not for floating-point or SIMD instructions. STOKE’s [3] improvements include complex optimizations, but the tool relies on randomized testing and not formal verifications for its transformations. Therefore, the combined challenge of performing complex vector or floating point optimizations, formally verifying the transformation and integrating those capabilities into production compilers remained unsolved, until Minotaur.

Designing an efficient and high performing superoptimizer included several challenges. First, the search space of replacement options for a suboptimal code sequence is enormous, making an exhaustive search for the optimal choice computationally expensive. Second, the semantic formal and not random equivalency check adds further overhead – especially for hard to verify SIMD and floating point operations. Minotaur relies on SMT solvers for equivalency checks between the original code and the proposed optimization, but these solvers often struggle with large complex expressions. At the same time a sufficiently large number of lines of code is needed to provide enough context to be able to find improvement options in the first place. Furthermore, since Minotaur aims for low level, hardware aware optimizations, the authors used detailed information about pipelines and cache effects to accurately model the performance of optimizations. These difficulties of realizing high performance without too much computational cost, required a multiple stages process.

2.2 Contributions

Because today’s SMT solvers cannot optimize entire LLVM functions, Minotaur instead optimizes each instruction individually. It therefore breaks the LLVM function in SSA form down into cuts, which are small loop-free subsets of the the instruction’s dependencies, to capture where the values used in the instruction come from. The size of the cut hereby influences the optimization, as too small cuts contain not enough context to determine a better code sequence, while

too large cuts increase the solvers computation time dramatically. Through an empirical study the authors selected the cut size of four as it provides a good balance between finding most optimization while still running in a acceptable amount of time. For each cut, a set of potential alternatives which are faster are synthesized. These improvements are then verified with Alive2, which the authors extended to support SIMD operations. Lastly, for each optimization its speedup is estimated using a more accurate model (LLVM MCA) and its original and transformed code cached for reuse in future compilations.

To measure how Minotaur’s optimizations influence the overall performance of larger software projects, the authors benchmarked Minotaur on the SPEC CPU 2017, GMP and libYUV on Intel and AMD hardware with LLVM 18 compiled with -O3 and -march=native. They determined a maximum speedup of 1 - 7 percent, where integer SIMD fragments made out 75 percent of the discovered rewrites. When the compilation is performed with an empty cache, Minotaur increased compile times significantly. With a filled cache, however, only an additional 3 percent of computation time was needed. Overall, the authors determined 15 simple and general floating-point and vector optimizations, which they then reported the LLVM project.

3 Artifact Evaluation Results

TODO

4 Research Ideas

TODO

Acknowledgments

TODO

Availability

TODO

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. Minotaur: A SIMD-oriented synthesizing superoptimizer. *Proceedings of the ACM on Programming Languages (OOPSLA2)*, 8(OOPSLA2):1–25, October 2024. doi: 10.1145/3689766. <https://doi.org/10.1145/3689766>.
- [2] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer. *CoRR*, abs/1711.04422, 2017. URL <http://arxiv.org/abs/1711.04422>.
- [3] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. *CoRR*, abs/1211.0557, 2012. URL <http://arxiv.org/abs/1211.0557>.